

Vote → Kafka →
Worker →
PostgreSQL →
Result

Arquitectura de
Microservicios con DevOps
& Cloud Patterns

[github.com/BillyJoe121/microservices
-demo](https://github.com/BillyJoe121/microservices-demo)

MICROSERVICES DEMO

Workshop I

WHAT ARE WE BUILDING ON?

Real-time voting app (Tacos vs Burritos) with a multilingual microservices architecture deployed on Kubernetes / Okteto Cloud.

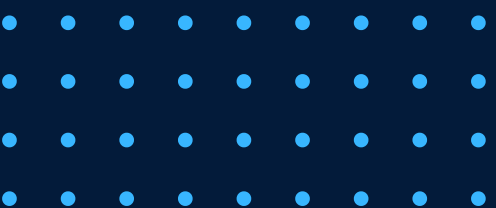
Vote Service — Java / Spring Boot

Worker — Go

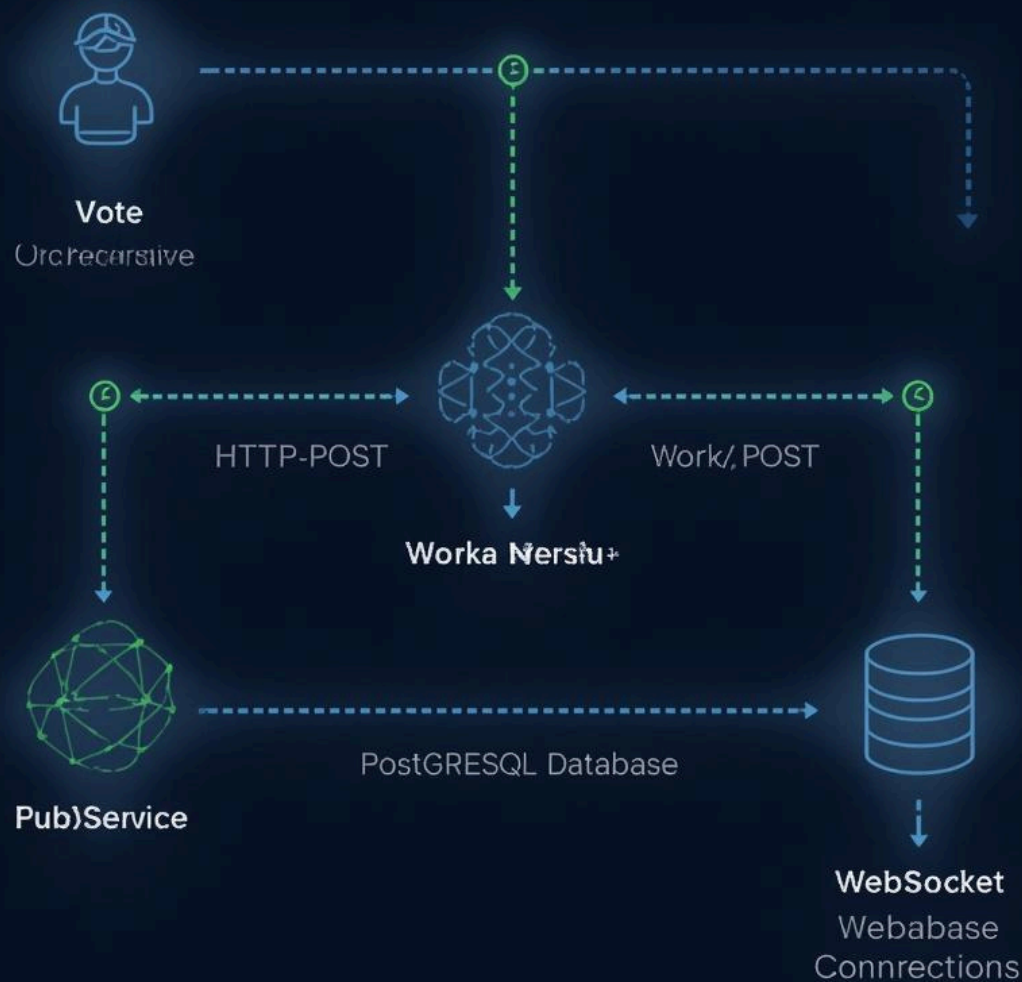
Result — Node.js / Express

PostgreSQL

Apache Kafka



SYSTEM ARCHITECTURE



Asynchronous Flow

Vote publishes in Kafka without blocking responses.

Real time

WebSocket delivers instant results to the user.

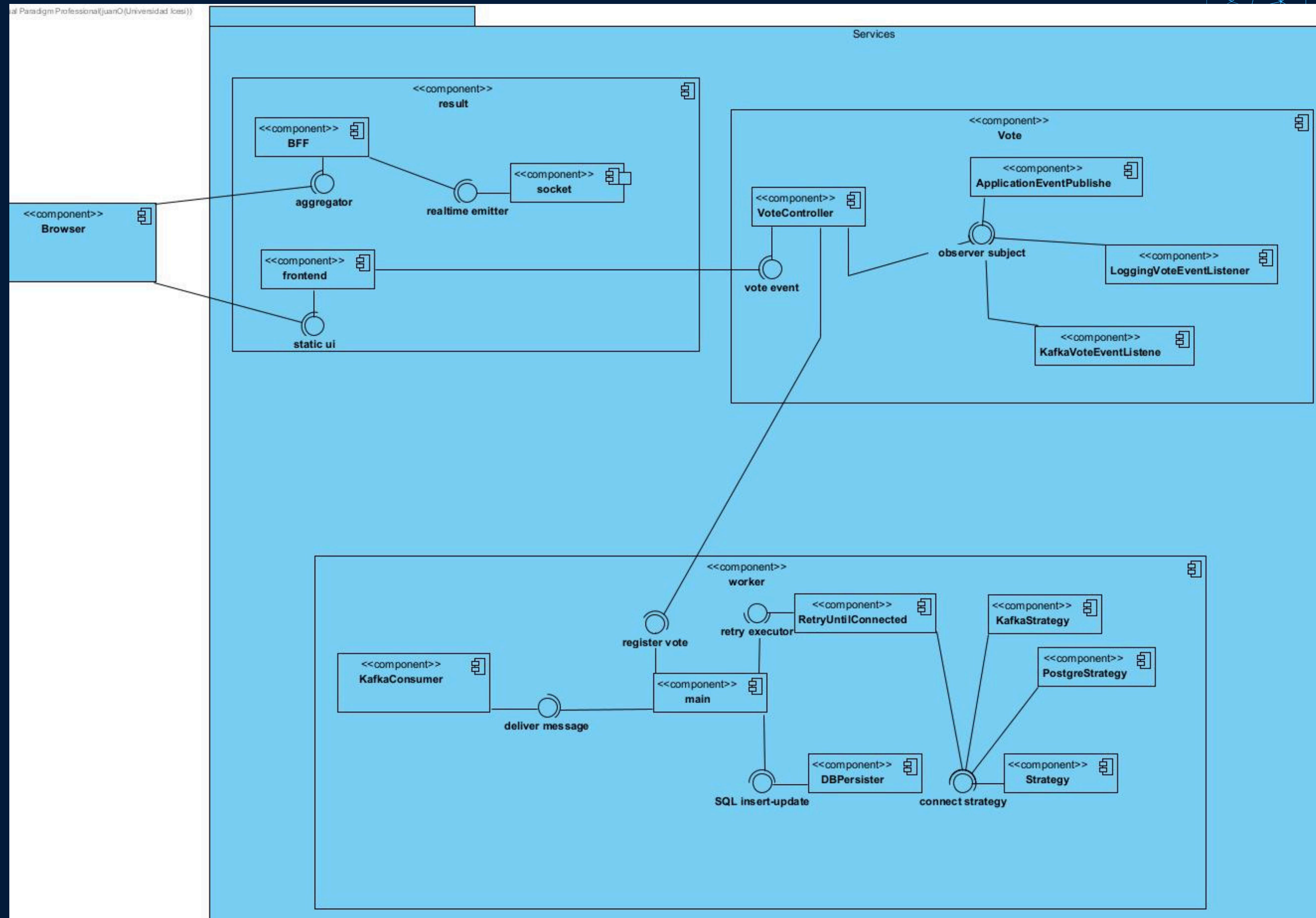
Decoupled

Independent services communicated via messaging.

Socket.io

Bidirectional client-server connection.

SYSTEM ARCHITECTURE



3 FOUND CLOUD DESIGN PATTERNS



Async Messaging



External
Config



Backends for FE

2 IMPLEMENTED SOFTWARE PATTERNS



Strategy

Java — Observer Pattern
(feature/vote-observer-pattern branch)

Before: Duplicate loops
Now: Generic and interchangeable function



Observer

```
getHandlers().forEach(handler ->  
    handler.onVoteReceived(voter, vote));
```

/ KafkaVoteHandler, LoggingVoteHandler

Before: the handler did everything
Now: it notifies registered observers

GITHUB FLOW – ALWAYS DEPLOYABLE MAIN

Each merge via PR

+ Code Review
+ Green CI

Branches:

- main (protected)

- feature/

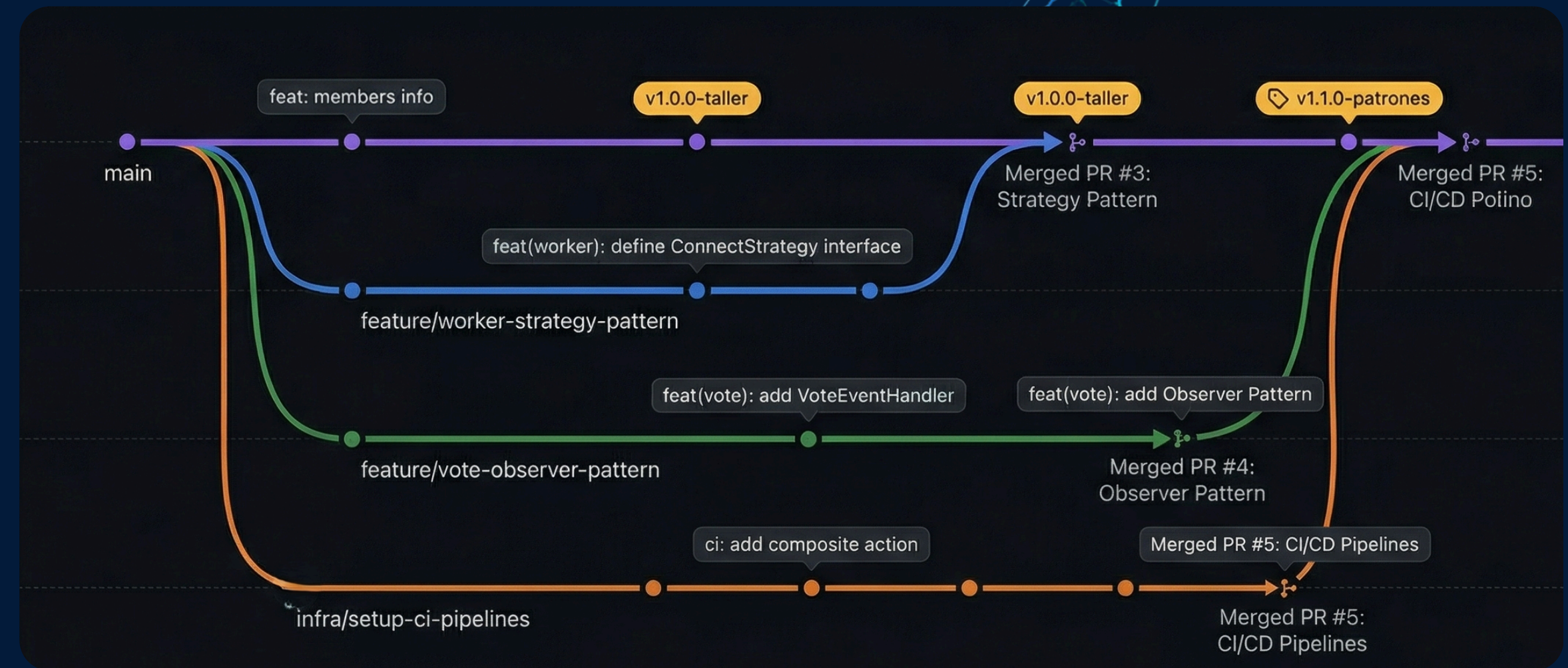
- infra/

- hotfix/

Semantic tags:

- v1.0.0-workshop

- v1.1.0-patterns





3 INDEPENDENT CI PIPELINES + 1 UNIFIED CD



Path Filtering

It is activated only if its
microservice changes
(paths: vote/**).



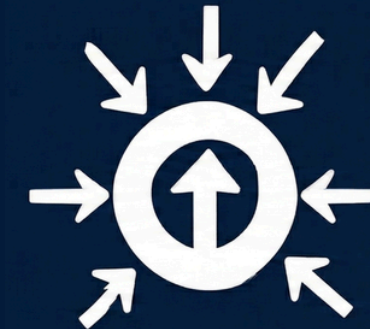
Composite Action

Build + Scan to a
reusable file (DRY).



Shift-Left Security

Trivy Scan rejects
image if there is a
critical CVE.



CD Unificado

staging →
production with
approval.

INFRASTRUCTURE.YML – IAC AS FIRST-CLASS CODE

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - '2181:2181'
    networks:
      - micro-demo

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    ports:
      - '9092:9092'
    depends_on:
      - zookeeper
    networks:
      - micro-demo

  postgresql:
    image: postgres:13
    environment:
      - POSTGRES_USER=okteto
      - POSTGRES_PASSWORD=okteto
      - POSTGRES_DB=votes
    ports:
      - '5435:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
```



yamllint

Automatically validated
YAML syntax and structure.



helm lint

Validation of Kafka charts +
PostgreSQL.



hadolint

Anti-patterns detected in
Dockerfiles.



Checkov

IaC scanning vs CIS policies.

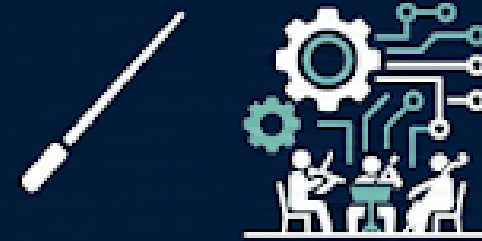


ZERO TRUST INFRASTRUCTURE



Development

`docker-compose.yml` ·
`okteto.yml` for local and
cloud dev environments.



Orchestration

Kustomize (own apps) ·
Helm (Kafka, PostgreSQL
third parties).



Security

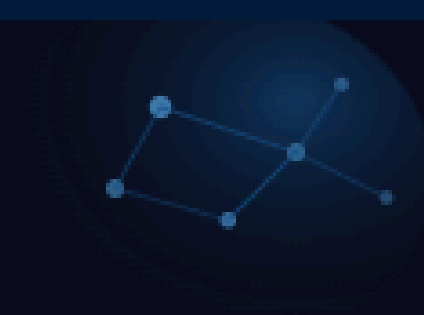
`runAsUser:10001` ·
`readOnlyRootFilesystem` ·
NetworkPolicy default-
deny.



Reliability

`livenessProbe` ·
`readinessProbe` ·
`PersistentVolumeClaim`.

WHAT DID WE ACHIEVE?



5 Pipelines

Vote · Worker · Result · CD · Infra



GitHub Flow

Mandatory PRs · Conventional Commits



0 Violations

Checkov + Yamllint on main



DevSecOps

Trivy + Checkov Shift-Left



5 Patterns

3 Cloud + 2 Software



DRY via Composite Actions

Reusable workflow components



3 Languages

Java · Go · Node.js



Zero Trust

non-root, immutable filesystem, default-deny network

DEMO